



**REPÚBLICA BOLIVARIANA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA: ingeniería informática
UNIDAD CURRICULAR: Lenguajes y Compiladores**

Tema 2: Los lenguajes de programación

NOMBRE DEL DOCENTE: Félix Márquez

NOMBRE DE ESTUDIANTES:

José Silva, V-30.810.283 SEC: 2

Nicole Serano, V-31.275.411 SEC: 2

Eloismary Díaz, V-28.682.879 SEC: 2

José Moreno, V-28.086.251 SEC: 2

CIUDAD GUAYANA, 03– 06 – 2026

Resumen Académico

Propósito: El documento tiene como objetivo estudiar y comparar la estructura léxica y sintáctica de lenguajes de programación modernos para analizar cómo convergen hacia arquitecturas híbridas. Busca demostrar cómo el diseño de un lenguaje influye en el equilibrio entre la facilidad de escritura, la gestión de memoria y el rendimiento de la computadora. Además, propone el diseño de un Lenguaje de Dominio Específico (DSL) llamado "ECO-L" para optimizar la supervisión y automatización de microrredes eléctricas inteligentes (ECO-GRID).

Lenguajes: En la sección comparativa se analizan detalladamente cuatro lenguajes de amplio uso comercial: Zig, Rust, Python y JavaScript. Posteriormente, el informe introduce y detalla la especificación gramatical y semántica de ECO-L, el lenguaje de dominio específico creado por los autores.

Resultados: Se determinó que la tendencia moderna (liderada por Zig y Rust) rechaza la herencia tradicional en favor de la composición, y que los lenguajes estáticos integran cada vez más características funcionales o declarativas avanzadas. En las pruebas reales de rendimiento mediante la evaluación matemática de la Conjetura de Collatz (de 1 a 5,000,000), los entornos compilados (AOT) demostraron una velocidad drásticamente superior: Zig obtuvo el mejor tiempo promedio con 3.393,21 ms, seguido por Rust con 4.564,67 ms. Por su parte, JavaScript (JIT Node.js) registró 28.817,14 ms y Python (interpretado) finalizó con el rendimiento más bajo al registrar 320.588,83 ms. Todos mantuvieron un consumo registrado de 0,00 MB. Se validó con éxito la viabilidad del lenguaje ECO-L a través del diseño formal de su alfabeto, reglas léxicas y gramática abstracta, demostrando su efectividad mediante la resolución práctica de dos escenarios operativos críticos: prevención de fuga térmica en baterías y balance autónomo de carga energética.

Introducción

Hoy en día, los lenguajes de programación ya no se limitan a una sola forma de trabajar, sino que mezclan lo mejor de diferentes estilos para adaptarse a lo que pide el mercado. En este informe vamos a estudiar y comparar cuatro lenguajes muy usados: Zig, Rust, Python y JavaScript. Aunque nacieron con ideas y propósitos muy diferentes, todos han evolucionado para buscar un equilibrio entre ser fáciles de escribir y aprovechar al máximo el poder de la computadora. A lo largo del documento, analizaremos cómo están contruidos, cómo manejan sus reglas de escritura y, finalmente, veremos una prueba real de su velocidad usando un problema matemático llamado la Conjetura de Collatz, demostrando cómo la estructura de un lenguaje cambia por completo su rendimiento.

Actividad 1: Marco Teórico Analítico: Convergencia De Paradigmas En Lenguajes

Modernos

1. Análisis De Convergencia

La evolución de la ingeniería de software ha demostrado que las fronteras rígidas entre paradigmas de programación resultan insuficientes para abordar las complejas demandas del mercado actual. Los lenguajes modernos analizados en el informe (Zig, Rust, Python y JavaScript) no se adhieren de forma exclusiva a una única doctrina de diseño. Por el contrario, convergen hacia arquitecturas híbridas orientadas a la optimización del rendimiento, la seguridad en memoria, la legibilidad y la escalabilidad de los sistemas.

Esta adopción simultánea de múltiples características de diseño (o "paradigmas cruzados") responde a exigencias concretas:

- La necesidad de abstracciones de alto nivel sin perder el control de bajo nivel del hardware (hardware micro-controlado o sistemas en la nube de alta concurrencia).
- El desarrollo iterativo rápido combinado con tipados robustos o mecanismos predictivos en tiempo de compilación.

Por ejemplo, Rust fusiona un núcleo estrictamente imperativo de control de memoria con abstracciones funcionales basadas en expresiones y validación algebraica de tipos. Python y JavaScript, nacidos bajo filosofías dinámicas orientadas a objetos y scripts, han incorporado elementos funcionales avanzados y estructuras declarativas (como "match/case") para el procesamiento de datos complejos. Zig, por su parte, reduce el lenguaje a un modelo imperativo/estructural minimalista pero inyecta capacidades de metaprogramación declarativa en tiempo de compilación (comptime) para eliminar la necesidad de macros complejas.

2. Paradigmas de programación

A continuación, se fundamentan y diseccionan los cinco paradigmas esenciales que configuran la estructura léxica y sintáctica de los entornos de desarrollo contemporáneos.

2.1 Paradigma Imperativo/Estructural

Este paradigma se centra en el "cómo" se debe ejecutar un programa, describiendo la computación en términos de sentencias que cambian el estado del sistema. Sus componentes críticos analizados son:

- **Gestión de Estado:** Consiste en la manipulación directa de las posiciones de memoria física o lógica a través de variables. En entornos como Zig y Rust, la gestión de estado incluye la delimitación explícita mediante palabras clave como 'var' / 'const' (Zig) o 'let' (Rust).

- **Mutabilidad:** La capacidad de alterar un valor asignado a una variable en el tiempo. Rust asume la inmutabilidad por defecto y exige la palabra clave explicativa 'mut' para habilitar la mutabilidad, mientras que lenguajes dinámicos tradicionales permiten mutabilidad libre, lo cual repercute en la previsibilidad del código.

- **Efectos Secundarios:** Cualquier cambio de estado observable fuera del entorno local de una función (como modificar una variable global o escribir en el disco). En el análisis de firmas de funciones, el uso de delimitadores estrictos (puntos y comas obligatorios) y bloques cerrados resguarda la secuencialidad de estos efectos secundarios.

2.2 Paradigma Orientado a Objetos (POO)

Organiza el código en unidades llamadas objetos, los cuales encapsulan datos y comportamientos.

Los lenguajes modernos muestran una evolución drástica de este paradigma:

- Encapsulamiento: La ocultación del estado interno de un objeto, exponiendo solo interfaces públicas. Sintácticamente, esto se logra mediante modificadores de visibilidad como 'pub' (en Zig y Rust) o estructuras formalizadas como 'class' (en Python y JavaScript).
- Polimorfismo: La propiedad por la cual diferentes objetos pueden responder al mismo mensaje o firma de función de formas distintas. En lenguajes estáticos como Rust, se gestiona mediante el sistema de "traits" (por ejemplo, 'Intolterator' para bucles 'for'), mientras que en Python o JavaScript se da por tipado dinámico o polimorfismo de subtipos.
- Herencia vs. Composición: La herencia permite que una clase derive de otra (soportada de forma tradicional en Python y JavaScript vía clases). Sin embargo, la tendencia moderna (marcada intensamente por Zig y Rust) es rechazar la herencia debido a la rigidez que introduce. En su lugar, se favorece la Composición a través de la combinación de estructuras limpias ('struct') y rasgos (traits o uniones de tipos), permitiendo acoplamientos débiles y mayor seguridad léxica.

2.3 Paradigma Funcional

Concibe la computación como la evaluación de funciones matemáticas y evita el cambio de estado y los datos mutables. Sus pilares analizados incluyen:

- Inmutabilidad: Una vez que un dato es creado, no puede ser alterado. Esto previene condiciones de carrera y simplifica el análisis estático. Se observa su adopción nativa en las constantes de Zig o el diseño por defecto de Rust.

- Funciones de Primer Orden: Significa que las funciones pueden pasarse como argumentos, devolverse desde otras funciones y asignarse a variables. Sintácticamente, se manifiesta en las expresiones anónimas como las funciones 'lambda' de Python, los 'closures' de Rust o las 'arrow functions' (funciones flecha) de JavaScript.

- Transparencia Referencial: Una función posee esta propiedad si, para un mismo argumento de entrada, produce siempre el mismo valor de salida sin causar efectos secundarios. El análisis sintáctico de Rust simula esto al forzar que las funciones orientadas a expresiones retornen valores de forma directa al bloque padre si se omite el punto y coma final.

2.4 Paradigma Lógico/Declarativo

Este paradigma se enfoca en el "qué" se quiere lograr o describir, abstrayendo el flujo de control explícito de la máquina.

- Unificación: Un mecanismo de emparejamiento de patrones que busca la consistencia entre estructuras de datos. En los lenguajes bajo estudio, la unificación ha mutado hacia potentes sentencias sintácticas de Pattern Matching. Ejemplos de esto son el bloque 'match' en Rust (donde el compilador valida formalmente la exhaustividad de los patrones) y el 'match/case' de Python.

- Abstracción del Flujo de Control: El programador no diseña los saltos lógicos paso a paso (evitando los riesgos de mecanismos como el fall-through del 'switch' tradicional en C o JavaScript). En lugar de ello, el Árbol de Sintaxis Abstracta (AST) se encarga de resolver la jerarquía estructural de las condiciones basadas en la semántica de la coincidencia de patrones declarada por el usuario.

2.5 Paradigma Concurrente / Modelo de Actores

Atiende la ejecución simultánea de múltiples flujos de cómputo, un factor crucial dado que el rendimiento bruto mononúcleo ha alcanzado límites físicos (tal como demuestran las variaciones en los tiempos de ejecución de la Conjetura de Collatz).

- Paso de Mensajes: En lugar de compartir memoria para comunicarse (lo que induce a corrupción de datos), los hilos de ejecución o entidades independientes se comunican enviándose datos de manera explícita.
- Aislamiento de Estado: Cada unidad de ejecución posee su propio estado local inaccesible para otras. En el espectro analizado, JavaScript implementa esto a través de su arquitectura asíncrona no bloqueante conducida por un bucle de eventos (Event Loop), expresada sintácticamente mediante firmas 'async/await'. Por otro lado, la gramática estricta de Rust ('unsafe', 'Result', control de ciclo de vida) actúa como un validador sintáctico para garantizar la concurrencia segura libre de carreras de datos, aislando los hilos o forzando transferencias seguras de propiedad de la memoria.

Actividad 2: Estudio Comparativo Morfológico (Léxico) y Sintáctico

1. Análisis Morfológico (Léxico)

El análisis morfológico es la primera fase de la compilación/interpretación, donde el flujo de caracteres de entrada se agrupa en secuencias con significado propio llamadas tokens (componentes léxicos). A continuación, se detalla el comportamiento de cada lenguaje frente a los elementos léxicos más significativos:

1.1 Manejo de Tokens y Palabras Reservadas

Cada uno de los entornos utiliza un conjunto particular de palabras reservadas (keywords) que no pueden utilizarse como identificadores:

- Zig: Tiene una filosofía minimalista y estricta ("solo una forma de hacer las cosas").

Cuenta con 36 palabras clave (ejemplo: ``fn``, ``pub``, ``comptime``, ``defer``, ``errdefer``, ``const``, ``var``, ``struct``). Destacan palabras orientadas a la programación en tiempo de compilación y control manual de recursos.

- Rust: Cuenta con alrededor de 53 palabras clave divididas en:

1) Estrictas (ejemplo: ``fn``, ``let``, ``match``, ``impl``, ``pub``, ``mut``, ``unsafe``).

2) Reservadas para el futuro fuertes (ejemplo: ``become``, ``priv``), y débiles (ejemplo: ``union``, ``static``).

3) La presencia de ``mut`` (para mutabilidad explícita) y ``unsafe`` define su filosofía de seguridad en memoria.

- Python: Es dinámico pero con un léxico estructurado poseyendo unas 35 palabras clave (ejemplo: ``def``, ``class``, ``import``, ``lambda``, ``with``, ``yield``, ``None``, ``True``, ``False``, ``pass``,

``nonlocal`, `global``). No utiliza palabras para declaración de tipos, sino para el control de ámbito y flujos abstractos.

- JavaScript: Cuenta con más de 40 palabras clave (ejemplo: ``function`, `let`, `const`, `var`, `class`, `import`, `export`, `async`, `await`, `yield`, `typeof`, `instanceof``).

1.2 Identificadores

Los identificadores son los nombres asignados a variables, funciones, clases o tipos.

Características	Zig	Rust	Python	JavaScript
Regla de formación	<code>[a-zA-Z_][a-zA-Z0-9_]*</code> o identificador arbitrario <code>@""</code>	<code>[a-zA-Z_][a-zA-Z0-9_]*</code> o crudos <code>r#</code>	<code>[a-zA-Z_][a-zA-Z0-9_]*</code> (soporta Unicode de forma nativa)	<code>[a-zA-Z_\$][a-zA-Z0-9_\$]*</code> (permite \$ e identificadores Unicode)
Estilo estándar	camelCase (funciones) / snake_case (variables)	snake_case estricto (compilador arroja warnings si no se cumple)	snake_case (según especificación PEP 8)	camelCase (estándar de la comunidad ECMAScript)
Identificadores crudos	<code>@ "nombre"</code> permite palabras reservadas en identificadores	<code>r#fn</code> permite usar palabras reservadas como variables	No soporta de forma estándar	No soporta (genera error de sintaxis instantáneo)

1.3 Literales

Los literales representan valores constantes insertados directamente en el código fuente.

Zig:

- Números: Decimal (123), binario (0b101), octal (0o755), hexadecimal (0xff).

Permite guiones bajos para lectura (1_000_000).

- Textos: Cadenas UTF-8 encerradas en comillas dobles "...". No existen cadenas multilinea tradicionales; en su lugar, utiliza literales de línea continuos precedidos por \\\.

Rust:

- Números: Permite sufijos de tipo explícitos (1_000_000_u64, 42_i32, 3.14_f64).
- Textos: Cadenas UTF-8 en "...". Cadenas "crudas" en r#"..."# (evita escapar caracteres). Literales de byte (b'A') y cadenas de bytes (b"hello").

Python:

- Números: Enteros de precisión arbitraria. Flotantes (3.14). Complejos (3 + 4j).
- Textos: Delimitados por comillas simples '...' o dobles "...". Triple comilla para cadenas multilínea ('''...' o ''''...''). Cadenas formateadas (f"..."), crudas (r"...") y de bytes (b"...").

JavaScript:

- Números: Soporta tipo Number (flotante de doble precisión IEEE 754) y BigInt (9007199254740991n).
- Textos: Delimitados por '...', '"...' y backticks `...` (plantillas de cadena con interpolación activa \${expresión} y multilínea nativa).

1.4 Delimitadores e Indentación

El tratamiento léxico del espaciado y fin de sentencia contrasta fuertemente las filosofías de diseño:

- Python (Indentación Significativa):

No utiliza llaves para agrupar bloques, sino el nivel de indentación (espacios o tabuladores, PEP 8 recomienda 4 espacios).

El analizador léxico inserta tokens lógicos especiales llamados INDENT y DEDENT cuando la sangría cambia, y NEWLINE al final de la línea física.

Los delimitadores de expresiones son los dos puntos (:) para iniciar bloques y la nueva línea para terminar sentencias (no requiere punto y coma ;).

- Zig: Utiliza llaves para bloques y requiere punto y coma (;) estrictamente al final de cada sentencia. Los espacios en blanco son completamente ignorados léxicamente (free-form).

- Rust: Utiliza llaves para bloques. El punto y coma (;) es semánticamente significativo: si una expresión termina en punto y coma (;), su valor de retorno es descartado (se convierte en la tupla vacía ()). Si no lleva (;), la expresión retorna su valor al bloque padre.

- JavaScript: Utiliza llaves para delimitar bloques. Soporta Inserción Automática de Puntos y Comas (ASI). El analizador léxico inserta automáticamente (;) bajo ciertas reglas al encontrar saltos de línea, lo que a veces introduce sutiles errores lógicos.

2. Análisis Sintáctico: Jerarquía de Estructuras de Control

El análisis sintáctico toma los tokens provistos por el analizador morfológico y construye un Árbol de Sintaxis Abstracta (AST), validando que el código respete la gramática formal del lenguaje. A continuación, documentamos la jerarquía y gramática de las estructuras de control (bucles, condicionales y subprogramas).

2.1 Jerarquía Comparativa de Sentencias de Control

A --> [Estructuras de Control]

B --> [Condicionales]

C --> [Bucles / Iteración]

D --> [Subprogramas / Funciones]

- Condicionales

B --> B1 [if / else]

B --> B2 [Selección Múltiple]

B1 --> |Es Expresión| B1 [Rust / Zig]

B1 --> |Es Sentencia| B1 [Python / JS]

B2 --> |match / case| B2 [Rust / Python]

B2 --> |switch| B2 [Zig / JS]

- Bucles

C --> C1 [Condicionales Básicos]

C --> C2 [Iteradores sobre Colección]

C --> C3 [Bucles Infinitos Nativos]

C1 -->|while| C1 [Todos los Entornos]

C1 -->|do-while| C1 [JavaScript]

C2 -->|for ... in / of| C2 [Rust / Python / JS]

C2 -->|for ... | C2 [Zig / JS estándar]

C3 -->|loop| C3 [Rust]

C3 -->|while true| C3 [Zig / Python / JS]

- Subprogramas

D --> D1 [Declarativos / Nombrados]

D --> D2 [Expresiones / Anónimos]

D1 -->|fn / pub fn| D1 [Rust / Zig]

D1 -->|def| D1 [Python]

D1 -->|function| D1 [JavaScript]

D2 -->|Closures / Arrow / Lambda| D2 [Rust / JS / Python]

2.2 Condicionales: Gramática y Filosofía

- Rust (“if/else” y “match”):

Filosofía: En Rust, casi todo es una expresión. El condicional if/else evalúa a un valor, lo que obliga a que todas las ramas devuelvan exactamente el mismo tipo si se usa como asignación.

Gramática Formal (EBNF Simplificado):

```
IfExpression = "if" Expression Block ("else" ( Block | IfExpression ))? ;
```

```
MatchExpression = "match" Expression { ( MatchArm )* } ;
```

```
MatchArm = Pattern ["if" Expression ] "=>" ( Expression , | Block ) ;
```

Particularidad: El compilador valida la “exhaustividad” del “match”, garantizando en tiempo de compilación que no existan casos sin cubrir.

- Zig (“if/else” y “switch”)

Filosofía: Similar a Rust, “ if ” es una expresión. Adicionalmente, cuenta con soporte directo para desenvolver tipos opcionales (?T) y uniones de error (Error!T) directamente en la cabecera del condicional.

Gramática Formal (EBNF Simplificado):

```
IfExpr = "if" ( " Expr " ) Expr ( "else" Expr )? ;
```

```
SwitchExpr = "switch" ( " Expr " ) { " ( SwitchProng )* } ;
```

```
SwitchProng = SwitchItem "=>" Expr , ;
```

- Python (“if/elif/else” y “match/case”)

Filosofía: Condicionales basados puramente en bloques de indentación. No son expresiones, por lo que no se pueden asignar directamente a variables a menos que se use la expresión condicional ternaria: `x = a if cond else b`.

Gramática Formal (EBNF Simplificado):

`IfStmt = "if" Expression : Suite ( "elif" Expression : Suite )* [ "else" : Suite ] ;`

`MatchStmt = "match" SubjectExpression : NEWLINE INDENT ( CaseBlock )+ DEDENT ;`

- JavaScript (“if/else” y “switch/case”)

Filosofía: Basado en la especificación sintáctica tradicional de C. Son sentencias. El bloque “switch” sufre de “fall-through” por defecto, requiriendo instrucciones explícitas de control (“break”) para evitar ejecutar los casos siguientes.

Gramática Formal (EBNF Simplificado):

`IfStatement = "if" ( " Expression " ) Statement [ "else" Statement ] ;`

`SwitchStatement = "switch" ( " Expression " ) { ( CaseClause | DefaultClause )* } ;`

2.3 Bucles: Gramática y Filosofía

- Rust (“loop”, “while”, “for”)

`loop`: Bucle infinito nativo. Es una expresión y puede retornar un valor utilizando “`break valor;`”.

`while`: Bucle condicional básico. También existe “`while let`” para desestructurar flujos de datos condicionales.

for: Itera exclusivamente sobre estructuras que implementan el trait “Intolterator”.

Sintaxis:

```
etiqueta: for elemento in iterador { ... }
```

- Zig (“while”, “for”)

En Zig, “while” permite una cláusula de continuación opcional ejemplo: “while (cond):
(expresion_incremento).

for: Itera sobre rangos o porciones de memoria (slices/arrays). Puede iterar de manera paralela sobre múltiples arrays de la misma longitud.

Sintaxis:

```
for (items, 0..) |item, index| { ... }
```

- Python (“while”, “for”)

Cláusula “else” en bucles: Una particularidad sintáctica única. El bloque “else” se ejecuta únicamente si el bucle termina de forma natural (es decir, sin haber encontrado una sentencia `break`).

Sintaxis:

```
for item in iterable:
```

```
    # código
```

```
else:
```

```
    # se ejecuta si no hubo break
```

- JavaScript (“while”, “do-while”, “for”, “for...in”, “for...of”)

for...in: Itera sobre las propiedades enumerables de un objeto (claves).

for...of: Itera sobre los valores de un objeto iterable (arrays, maps, strings).

Sintaxis:

```
for (const item of iterable) { ... }
```

2.4 Subprogramas (Funciones y Métodos)

- Rust: Programación Funcional y Tipado Estricto

Las funciones se declaran con “fn”.

Signatura: Requiere tipos explícitos para todos sus parámetros y tipo de retorno (excepto si retorna “()”).

Gramática EBNF:

```
Function = [ "pub" ] [ "const" ] [ "unsafe" ] "fn" Identifier [ Generics ] ( Params ) [ -> Type ] Block
;
```

- Zig: Comptime y Control de Errores Integrado

Las funciones no tienen sobrecarga, pero pueden evaluarse en tiempo de compilación anteponiendo “comptime”.

Su firma permite retornos del tipo “anyerror!T” (unión de errores).

Gramática EBNF:

`FnProto = [ "pub" ] "fn" Identifier ( Params ) ( "!" )? Type Block ;`

- Python: Tipado Dinámico y Decoradores

Las funciones se declaran con “def”.

Soporta anotaciones de tipo (Type Hints) desde Python 3.5, que son ignoradas en ejecución por el intérprete pero validadas por linters estáticos como “mypy”.

Permite valores por defecto, argumentos posicionales variables (“*args”) y argumentos de clave variable (“**kwargs”).

Gramática EBNF:

`FunctionDef = [ Decorators ] "def" Identifier ( [ Parameters ] ) [ "->" Expression ] : Suite ;`

- JavaScript: Múltiples Paradigmas y Cláusulas

Las funciones tradicionales se declaran con “function”.

Las funciones flecha (`const f = () => {}`) no tienen su propio enlace a “this” ni al objeto “arguments”, heredándolos léxicamente del ámbito superior.

Soporta generadores (“function*”), clausuras dinámicas y es inherentemente asíncrono a través de firmas “async function”.

Gramática EBNF:

`FunctionDeclaration = [ "async" ] "function" [ "*" ] Identifier ( [ FormalParameters ] ) { "`

`SourceElements " } ;`

3. Síntesis Comparativa Morfológica y Sintáctica

Dimensión	Zig	Rust	Python	JavaScript
Fin de sentencia	“;” Obligatorio	“;” Obligatorio (salvo en expresiones finales)	Nueva línea (o “;” opcional)	“;” Opcional (ASI - inserción automática)
Agrupación Bloques	Llaves “{” “}	Llaves “{” “}	Indentación física constante	Llaves “{” “}
Paradigma de sentencias	Híbrido (Sentencias y Expresiones)	Orientado a Expresiones (casi todo retorna valor)	Orientado a Sentencias	Orientado a Sentencias
Metaprogramación	“comptime” nativo en el AST	Macros declarativos e higiénicos por tokens	Decoradores y Metaclases dinámicas	Proxy, Reflect y `eval` dinámico
Manejo de errores	Valores de retorno explícitos (Error Unions)	Tipo algebraico “Result<T, E>” monádico	Excepciones estructuradas (try/except)	Excepciones dinámicas (try/catch)
Tipado léxico	Estático y explícito	Estático con inferencia de tipos	Dinámico estricto	Dinámico débil (coerción implícita)

4. Resultados de Rendimiento: Conjetura de Collatz (1 a 5,000,000)

Estudio comparativo formal bajo condiciones controladas (5 ejecuciones de promedio por entorno).

Lenguaje	Tipo de entorno	Tiempo de Ejecución Promedio	Consumo de Memoria
Rust	Compilado (AOT)	4.564,67 ms	0,00 MB
Zig	Compilado (AOT)	3.393,21 ms	0,00 MB
JavaScript	JIT (Node.js)	28.817,14 ms	0,00 MB
Python	interpretado	320.588,83 ms	0,00 MB

Actividad 3: Creación del lenguaje L

Diseño de un Lenguaje de Dominio Especifico (DSL), para Sistemas Lógico-Físicos Críticos (ECO-GRID)

1. Nombre del lenguaje

Lenguaje L: **ECO-L**

2. Descripción General

ECO-L es un Lenguaje de Dominio Específico (DSL) diseñado para la supervisión y control de sistemas de microrredes eléctricas inteligentes (ECO-GRID), este permite que los operadores gestionen de forma sencilla los recursos energéticos del sistema mediante instrucciones.

Permite a operadores industriales:

- Monitorear baterías.
- Leer sensores de temperatura
- Consultar generación solar/eólica.
- Controlar relés de potencia.
- Automatizar respuestas ante eventos críticos.
- Gestionar el almacenamiento energético.

3. Especificación del Alfabeto y reglas léxicas

Letras (Alfabeto latino)	A-Z a-z		
Dígitos	0-9		
Símbolos Especiales	() { } [] , ; = > < >= <== != + - * / _		
Operadores	Relacionales: • > • < • >= • <=	Lógicos: • Y • O • NO	Asignación: • =

	•	==		
	•	!=		

Identificadores

Representan dispositivos o variables, y deben comenzar por una letra.

Regla	Ejemplos
Letra (Letra Numero _) *	Batería_01 sensor_temp línea_Solar sector_Industrial

Literales Numéricos

Enteros	Decimales
55 10 100 90 20 1500	55.5 18.75 20.5 54.75

Booleanos

verdadero	Falso
-----------	-------

Comentarios

Regla	Ejemplo
# comentario	# Temperatura máxima permitida

Palabras reservadas del lenguaje

Inicialización	init_grid fin_grid
Variables	definir
Lectura de Sensores	leer_temperatura() leer_carga() leer_generacion()

	leer_flujo() leer_hora()
Gestión de Baterías	estado_carga() cargar_bateria() descargar_bateria()
Control de Relés	conmutar_linea() activar_rele() desactivar_rele()
Refrigeración	activar_refrigeracion() desactivar_refrigeracion()
Condicionales	si entonces sino fin_si
Ciclos	mientras ejecutar fin_mientras
Alarmas	emitir_alerta()

Gramática Sintáctica Abstracta

Programa principal	<Programa> ::= init_grid <Sentencia> fin_grid
Declaracion de variable	<variable> ::= definir identificador = valor Ejemplo: <i>definir temperatura = 55;</i>
Lectura de sensores	<lectura> ::= identificador = leer_temperatura(dispositivo); <lectura> ::= identificador = leer_carga(dispositivo); <lectura> ::= identificador = leer_generacion(dispositivo);
Condicional	si <i>condición</i> entonces <i>acciones</i> sino <i>acciones</i> fin_si
Bucle	mientras <i>condición</i> ejecutar

	<i>acciones</i> fin_mientras
--	-------------------------------------

Escenario Operativo A

Prevención de Fuga Térmica y Gestión de Alivio de Carga

Objetivo:	Código
Monitorear continuamente una batería y actuar automáticamente cuando la temperatura exceda el límite de seguridad.	init_grid definir <i>temperatura</i> = 0; mientras verdadero ejecutar <i>temperatura</i> = leer_temperatura(bateria_01); si <i>temperatura</i> > 55 entonces activar(refrigeracion_auxiliar); desconectar(linea_solar); conectar(red_comercial_respaldo); emitir_alerta("peligro: fuga termica"); sino emitir_alerta("sistema operando normal"); fin_si fin_mientras fin_grid
Explicacion	a. El sistema inicia la microred. b. Lee continuamente la temperatura de la batería. c. Si supera 55°C: • Activa refrigeración.

- Detiene la carga proveniente de paneles solares.
 - Conecta la red comercial.
 - Emite alertas de emergencia.
- d. El monitoreo continúa permanentemente.

Escenario Operativo B

Balance de Carga y Optimización Energética Autónoma

Objetivo:	Código
Administrar automáticamente el excedente energético y proteger las cargas críticas.	<pre> init_grid definir carga = 0; definir generacion = 0; definir hora = 0; carga = leer_carga(bateria_01); generacion = leer_generacion(paneles_solares); hora = leer_hora(); si carga > 90 y generacion > leer_demanda(red_local) entonces inyectar_red(excedente_energetico); emitir_alerta("venta de energia activada"); sino si carga < 20 y hora >= 18 entonces aislar(sector_administrativo); aislar(iluminacion_externa); conectar(area_medica); conectar(servidores_criticos); emitir_alerta("modo ahorro critico"); fin_si </pre>

	fin_si
	fin_grid
Explicacion a. Caso 1: Excedente energético Si: • La batería posee más del 90% de carga. • La generación solar supera la demanda. Entonces: • El sistema vende energía a la red pública. • Se genera un ingreso económico por excedentes. b. Caso 2: Déficit energético Si: • La batería baja del 20%. • Son horas nocturnas de alta demanda. Entonces: • Se desconectan sectores no esenciales. • Se preserva la energía para: ✓ Área médica. ✓ Servidores críticos. ✓ Sistemas prioritarios.	

4. Ventajas del DSL ECO-L

- Sintaxis simple y legible.
- Orientado a operadores industriales.
- Reduce errores de configuración.
- Permite automatización de eventos críticos.
- Facilita el monitoreo de sistemas energéticos inteligentes.

5. Conclusión

El DSL ECO-L fue diseñado específicamente para el entorno ECO-GRID, proporcionando una interfaz hombre-máquina clara para operadores de microrredes inteligentes. Su estructura léxica utiliza palabras reservadas intuitivas, mientras que su gramática sintáctica reduce la complejidad del control

industrial. Los programas desarrollados demuestran la capacidad del lenguaje para gestionar eventos críticos de seguridad térmica y optimizar el flujo energético de manera autónoma, garantizando confiabilidad, eficiencia y facilidad de interpretación por parte de operadores humanos y sistemas de compilación o interpretación.

El lenguaje ECO-L cumple los requisitos establecidos al definir:

- Alfabeto formal
- Reglas léxicas
- Palabras reservadas obligatorias
- Gramática sintáctica abstracta
- Escenario A completamente implementado
- Escenario B completamente implementado
- DSL orientado al dominio ECO-GRID de microredes inteligentes.

Conclusión

Al final, este análisis nos demuestra que no existe un lenguaje de programación mejor que otro, sino que cada uno está diseñado para priorizar cosas distintas. Por un lado, Zig y Rust tienen reglas más estrictas y requieren más detalle al escribir, pero a cambio ofrecen una velocidad impresionante y un control total de la memoria, ideal para sistemas pesados. Por el otro, Python y JavaScript prefieren darle comodidad al programador para escribir código rápido, dejando que sus motores internos se encarguen del trabajo pesado, aunque eso signifique sacrificar algo de velocidad. Como futuros ingenieros informáticos, entender estas diferencias nos permite elegir con criterio la herramienta exacta que necesita cada proyecto según sus límites y objetivos.

Bibliografía

- Zig Software Foundation. (2024). Zig Language Reference (v0.13.0).
<https://ziglang.org/documentation/0.13.0/>
- The Rust Project Developers. (2024). The Rust Reference. Rust Foundation. <https://doc.rust-lang.org/reference/>
- Python Software Foundation. (2025). The Python Language Reference (v3.12).
<https://docs.python.org/3/reference/>
- Van Rossum, G., Warsaw, B., & Coghlan, N. (2001). PEP 8: Style guide for Python code. Python Software Foundation. <https://peps.python.org/pep-0008/>
- ECMA International. (2024). ECMA-262: ECMAScript 2024 Language Specification (15th ed.).
<https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>

- Sebesta, R. W. (2019). Concepts of programming languages (12th ed.). Pearson. (Referencia complementaria ideal para la sección de análisis de convergencia, gramáticas EBNF y paradigmas imperativo, funcional y POO).